



Module Cloud

Évaluation pratique finale

École Hexagone

Examen Pratique Cloud fil rouge – Assistant documentaire Lite RAG

Durée : 3h

Travail : en binôme ou trinôme

Contexte

Dans ce TP final, vous devez construire une petite application IA capable de répondre à des questions à partir d'un document fourni.

Le document de travail est un texte en français : **discours de Barack Obama de 2013**. Ce texte est utilisé uniquement comme corpus documentaire pour tester votre application. Le TP ne porte pas sur une analyse politique du discours, mais sur la construction technique d'une chaîne RAG légère.

L'objectif est de produire une application simple, testable et documentée, en utilisant des outils open source et des services cloud gratuits ou accessibles gratuitement dans le cadre du TP.

À la fin du travail, un utilisateur doit pouvoir :

- charger ou sélectionner le document fourni ;
- indexer le contenu du document ;
- poser une question en langage naturel ;
- obtenir une réponse générée à partir du document ;
- visualiser les passages du document utilisés comme sources.

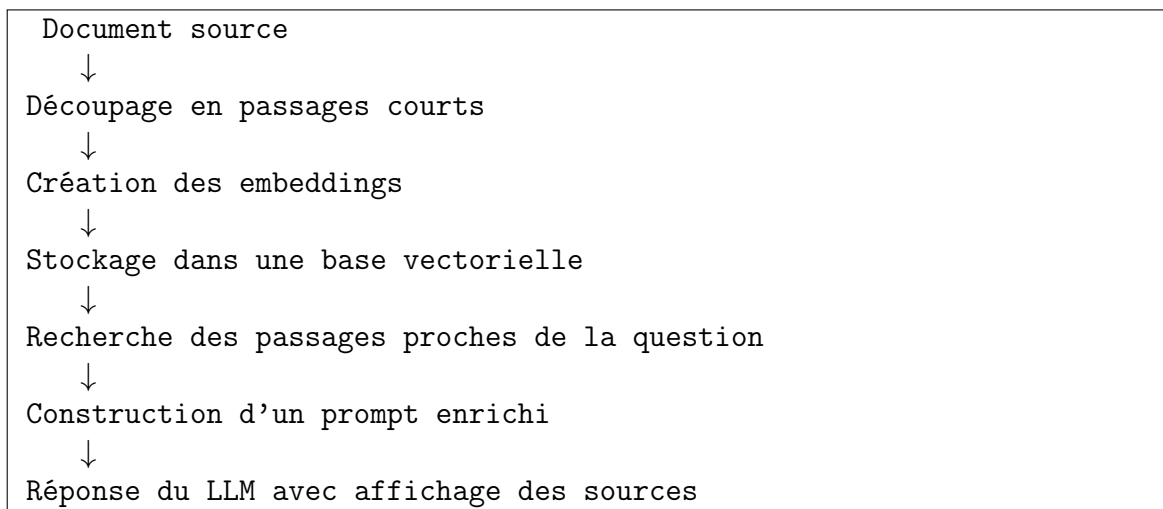
Le sujet est volontairement limité pour tenir dans une durée de 3h. Il ne s'agit pas de construire une application de production, mais une première application fonctionnelle illustrant les principes d'un RAG.

Rappel : qu'est-ce qu'un RAG ?

Un modèle de langage répond généralement à partir de ses connaissances internes. Il peut produire une réponse fluide, mais il ne connaît pas forcément les documents spécifiques que l'on souhaite exploiter.

Le principe du **RAG**, pour *Retrieval-Augmented Generation*, consiste à enrichir la réponse du modèle avec des passages extraits d'un document.

Le fonctionnement peut être résumé ainsi :



Dans ce TP, l'assistant ne doit pas se comporter comme un chatbot généraliste. Il doit répondre en s'appuyant sur les passages retrouvés dans le document.

Si l'information demandée n'est pas présente dans le document, l'application doit l'indiquer clairement.

Services et outils recommandés

Vous êtes libres dans l'organisation de votre code. Il n'est pas demandé de respecter une arborescence imposée.

Les outils suivants sont recommandés pour réaliser le TP :

Besoin	Outil recommandé
Environnement cloud de développement	GitHub Codespaces
Interface utilisateur	Streamlit
Backend API	FastAPI
Stockage du document brut	MinIO
Base vectorielle	ChromaDB
Orchestration RAG	LangChain
Modèle d'embeddings	sentence-transformers/paraphrase-multilingual-MiniLM-L12.5-v2
LLM local léger	Ollama avec qwen2.5:0.5b
Automatisation CI	GitHub Actions
Versioning et remise	GitHub

Vous pouvez utiliser une alternative si vous la justifiez clairement dans votre documentation.

Exemples :

- remplacer MinIO par un stockage local pour une version simplifiée ;
- utiliser un autre modèle léger disponible via Ollama ;
- construire une application Streamlit plus simple sans backend séparé ;
- utiliser une autre base vectorielle si son rôle est clairement expliqué.

Installation et test d'Ollama (à titre indicatif)

Ollama permet d'exécuter localement un petit modèle de langage, sans dépendre d'une API externe payante.

Dans le terminal de votre environnement Codespaces, installez Ollama :

```
curl -fsSL https://ollama.com/install.sh | sh
```

Démarrez ensuite le serveur Ollama :

```
ollama serve
```

Si ce terminal reste occupé, ouvrez un second terminal.

Dans le second terminal, téléchargez le modèle recommandé :

```
ollama pull qwen2.5:0.5b
```

Vous pouvez tester le modèle avec :

```
ollama run qwen2.5:0.5b
```

Exemple de question de test :

```
Explique en une phrase ce qu'est un RAG.
```

Le modèle `qwen2.5:0.5b` est volontairement léger. Il peut être moins précis qu'un grand modèle, mais il est suffisant pour démontrer le fonctionnement d'une chaîne RAG simple.

Travail demandé

Vous devez construire une application capable d'exploiter le document fourni et de répondre à des questions à partir de ce document.

L'application doit couvrir les fonctionnalités suivantes.

1. Chargement du document

L'utilisateur doit pouvoir charger ou sélectionner le fichier fourni depuis l'interface.

Le format recommandé est `.txt`. Le fichier `txt` du sujet est fourni en pièce jointe de ce sujet. Le support PDF est accepté en bonus, uniquement si le texte est correctement extractible. Attention à utiliser un PDF avec un petit volumes de pages vu la qualité minimaliste des modèles d'embedding et LLM choisis.

Le document peut être stocké dans MinIO ou dans un stockage équivalent. Si vous ne mettez pas en place MinIO, vous devez expliquer votre choix dans le README.

2. Indexation du document

Votre application doit permettre d'indexer le document.

L'indexation consiste à :

- lire le contenu du document ;
- découper le texte en passages courts ;
- transformer chaque passage en embedding ;
- stocker les passages vectorisés dans une base vectorielle ;
- conserver des métadonnées simples : nom du fichier, numéro du passage, extrait du texte.

Paramétrage conseillé :

`chunk_size` : entre 700 et 900 caractères
`chunk_overlap` : entre 100 et 150 caractères
`top_k` : entre 3 et 4 passages

Ces valeurs peuvent être ajustées si vous expliquez votre choix.

3. Question utilisateur

L'utilisateur doit pouvoir poser une question depuis l'interface.

Exemples de questions possibles :

- Quelles priorités économiques sont évoquées dans le discours ?
- Que dit le texte sur l'éducation ?
- Quels passages parlent de l'énergie ou du climat ?
- Que propose le discours concernant le salaire minimum ?
- Quels éléments concernent la classe moyenne ?

4. Recherche documentaire

Pour chaque question, votre application doit :

- transformer la question en embedding ;
- rechercher les passages les plus proches dans la base vectorielle ;
- récupérer les passages pertinents ;
- construire un contexte documentaire pour le LLM.

5. Génération de réponse

Votre application doit envoyer au LLM un prompt contenant :

- la question de l'utilisateur ;
- les passages retrouvés ;
- une consigne demandant de répondre uniquement à partir du contexte.

Exemple de prompt possible :

```
Tu es un assistant documentaire.  
Réponds uniquement à partir du contexte fourni.  
Si l'information n'est pas présente dans le contexte, réponds :  
"Je ne trouve pas cette information dans le document fourni."  
  
Contexte :  
{context}  
  
Question :  
{question}  
  
Réponse :
```

La réponse doit être claire, concise et fondée sur le document.

6. Affichage des sources

L'application doit afficher :

- la réponse générée ;
- les passages utilisés comme sources ;
- le nom du document ;
- le numéro du passage ou un identifiant équivalent ;
- éventuellement le score de similarité.

Cela permet de vérifier que la réponse provient réellement du document.

Interface et backend

Vous devez exposer une application utilisable depuis un navigateur.

L'interface doit au minimum permettre :

- le chargement ou la sélection du document ;
- le lancement de l'indexation ;
- la saisie d'une question ;
- l'affichage de la réponse ;
- l'affichage des sources.

Si vous utilisez FastAPI, vous pouvez par exemple prévoir les endpoints suivants :

```
GET /health
POST /upload
POST /index
POST /ask
```

Ces endpoints ne sont pas obligatoires si vous réalisez une version simplifiée avec Streamlit uniquement. En revanche, votre code doit rester compréhensible, testable et documenté.

Déploiement et lien de test

Votre application doit être testable par le professeur.

La solution recommandée est d'utiliser GitHub Codespaces et de rendre public le port utilisé par Streamlit.

Exemple de lancement Streamlit :

```
streamlit run app.py -server.port 8501 -server.address 0.0.0.0
```

Si votre projet utilise un backend FastAPI séparé :

```
uvicorn main:app -host 0.0.0.0 -port 8000 -reload
```

Dans Codespaces, vous devez vérifier que le port de l'application est bien accessible depuis le navigateur.

Le lien de test de l'application devra être transmis au professeur.

GitHub Actions

Votre repository doit contenir une automatisation simple avec GitHub Actions.

Il n'est pas demandé de mettre en place un pipeline de production. L'objectif est de montrer que vous savez intégrer une première étape de CI dans un projet cloud/IA.

Votre workflow peut par exemple :

- installer les dépendances Python ;
- exécuter quelques tests simples ;
- vérifier qu'une fonction clé ne casse pas ;
- tester un endpoint de santé si vous utilisez FastAPI.

Exemples de tests utiles :

- test de découpage d'un texte en chunks ;
- test de construction du prompt ;
- test de disponibilité d'un endpoint `/health` ;
- test simple sur le format de réponse retourné.

Documentation attendue

Votre repository doit contenir un fichier `README.md` clair.

Le README doit expliquer :

- l'objectif du projet ;
- les outils utilisés ;
- la manière de lancer l'application ;
- la manière d'installer ou lancer Ollama ;
- la manière d'indexer le document ;
- un exemple de question ;
- les limites connues de l'application.

Vous devez également ajouter une courte description du fonctionnement RAG.

Exemple minimal :

Document -> Découpage -> Embeddings -> Base vectorielle -> Recherche
-> Prompt enrichi -> Réponse + sources

Livrables à transmettre

À la fin du TP, chaque groupe doit transmettre :

1. le lien du repository GitHub ;
2. le lien de test de l'application IA ;
3. une capture d'écran de l'application ;
4. le README du projet ;
5. une preuve que GitHub Actions s'exécute ;
6. un exemple de question/réponse avec les sources affichées.

Le professeur doit pouvoir comprendre, lancer ou tester le projet à partir des éléments fournis.

Critères d'évaluation

L'évaluation portera principalement sur :

- le fonctionnement général de l'application ;
- la bonne compréhension du principe RAG ;
- la qualité du découpage documentaire ;
- l'utilisation d'une base vectorielle ;
- la capacité à générer une réponse fondée sur les passages retrouvés ;
- l'affichage clair des sources ;
- la lisibilité du code ;
- la qualité du README ;
- l'usage pertinent des services cloud gratuits ;
- la présence d'une automatisation simple avec GitHub Actions.

Le design visuel de l'application n'est pas prioritaire. Une interface simple mais fonctionnelle vaut mieux qu'une interface travaillée mais instable.

Bonus possibles

Les bonus sont facultatifs. Ils ne doivent être réalisés que si le socle principal fonctionne déjà.

Exemples de bonus :

- prise en charge d'un PDF non scanné ;
- gestion de plusieurs documents ;
- bouton permettant de réinitialiser l'index vectoriel ;
- réglage du nombre de passages récupérés ;
- affichage du score de similarité ;
- meilleure mise en forme des sources ;
- dockerisation plus complète ;
- tests unitaires plus nombreux ;
- comparaison entre deux modèles Ollama légers.

Limites acceptées

Cette application est volontairement une version légère.

Les limites suivantes sont acceptées :

- temps de réponse parfois long ;
- qualité imparfaite du petit modèle ;
- corpus limité à un seul document ;
- absence d'authentification ;
- absence de monitoring avancé ;
- absence de déploiement production ;

— interface simple.

En revanche, le principe suivant doit être respecté :

L'assistant doit répondre à partir du document. S'il ne trouve pas l'information dans le document, il doit le dire clairement.

Résultat attendu

Le résultat attendu est une mini-application IA complète et testable.

Vous devez être capables d'expliquer :

- le rôle du document source ;
- le principe du découpage en passages ;
- le rôle des embeddings ;
- le rôle de la base vectorielle ;
- le rôle du LLM ;
- le chemin parcouru par une question utilisateur ;
- la manière dont les sources sont affichées.

Le but du TP n'est pas d'obtenir le meilleur modèle possible. Le but est de comprendre comment assembler une première application RAG, la rendre testable dans un environnement cloud gratuit, documenter les choix techniques et transmettre un code réutilisable.